ANDIYA MEDIA PLAYER

How this app is created and how it works

A guided internals document for developers who already know C/C++, C# or Python, but have not used Qt or QML yet. Real code from Andiya is mixed with small teaching examples.

Application: Andiya (Qt 6.8, C++20, QML)

Audience: C / C++ / C# / Python developers

Companion to the source tree at D:\Code\Andiya

Created for study of this codebase — not a substitute for official Qt documentation

Contents

1. What you are looking at
2. Map Qt onto languages you already know
3. The Qt object model: QObject, properties, signals
4. QML in one sitting (think XAML, not HTML)
5. Andiya's folder map and build
6. Boot sequence: from `main()` to the window
7. The window, theme, and reusable components
8. Playback: files, streams, frames, audio
9. The filter pipeline and plugin system
10. Library, captures, compare mode, workspace
11. Persistence, threading, and diagnostics
12. How to change the app yourself
13. Glossary and further reading

> **How to read this**
> Chapters 2–4 teach Qt/QML with small examples. Chapters 5–11 walk through Andiya specifically. Chapter 12 is a cookbook: add a button, a C++ property, a shortcut, a plugin. You can skip to 5 if you only want the app, then jump back when a QML line looks alien.

1. What you are looking at

Andiya is a desktop media player written in **C++20** with a **Qt Quick / QML** user interface. It plays video, audio and still images, opens network streams (RTSP, RTMP, HTTP(S) and similar), captures lossless PNG frames, compares original vs filtered frames, and runs third-party image filters as plugins.

It is not a WinForms app, not WPF, not Electron, and not a Python GUI. The closest mental picture if you come from .NET is:

- **C++ Qt objects** ≈ C# view-models / services (the "brain")
- **QML** ≈ XAML + a bit of JavaScript (the "look and layout")
- **Qt Multimedia / FFmpeg plugin** ≈ a media pipeline you do not write yourself
- **CMake** ≈ the project file (csproj / sln, but for C++)

The UI never talks to files or codecs directly. Buttons and sliders call methods on C++ objects that were registered at startup. Those objects emit signals when state changes, and QML bindings refresh automatically. That split is the whole architecture.

1.1 What the user sees, what the code owns

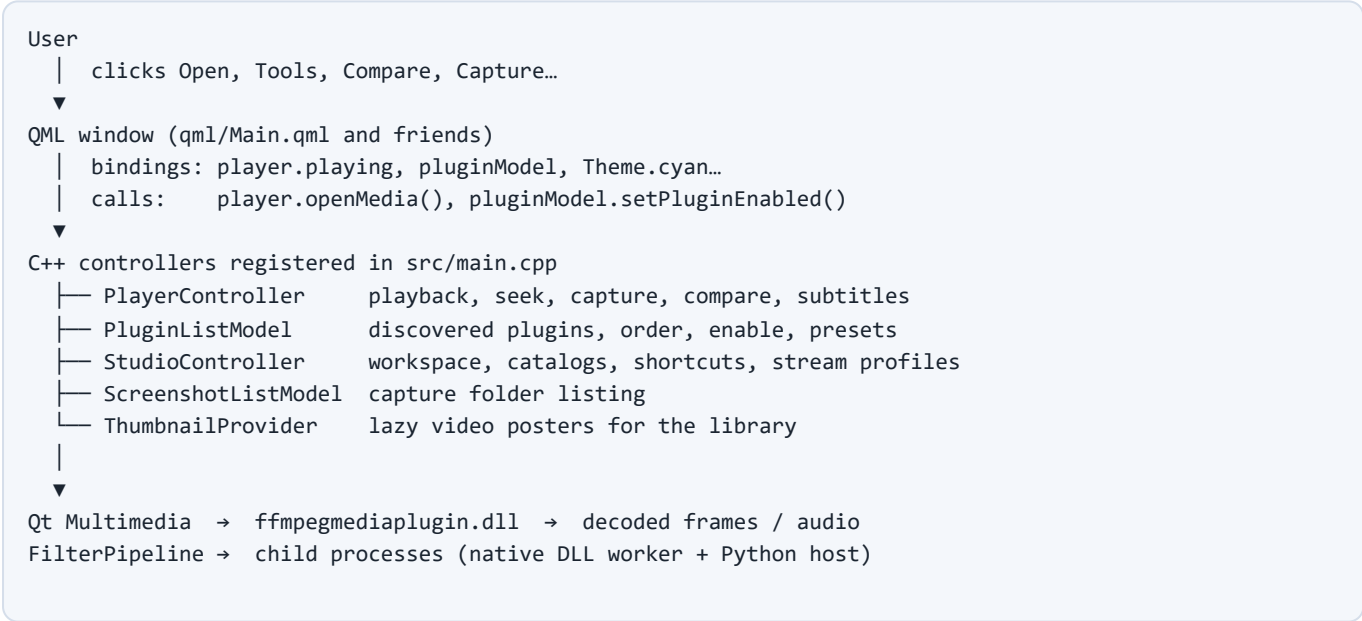


Figure 1. One-way ownership: QML presents state; C++ owns media, files, and plugins.

1.2 Technology stack (this tree)

Layer	Andiya uses	If you know…
Language	C++20	C# classes, Python classes, raw C
UI	Qt Quick / QML 6.8	WPF XAML, Flutter, React (declarative)
Widgets style	Qt Quick Controls 2, Basic style	We restyle controls ourselves
Media	Qt Multimedia + FFmpeg backend	NAudio + FFmpeg, MediaElement
Build	CMake 3.24+ / MSVC	MSBuild, dotnet build
Settings	QSettings	Properties.Settings, JSON config
Plugins	C ABI DLL + out-of-process Python	C# plugins + subprocess workers

Official Qt docs live at *doc.qt.io*. This PDF explains the subset Andiya actually uses, with analogies and with this repo's files.

2. Map Qt onto languages you already know

2.1 Qt is a C++ application framework

Qt is a large C++ library (plus tools) for writing desktop, mobile and embedded apps. “Qt” is not QML. QML is one UI technology that sits on top of Qt. You can write a Qt app with no QML at all (classic Widgets, like WinForms). Andiya chose Qt Quick/QML because it is better at custom, animated, GPU-friendly layouts — more like WPF or Flutter than WinForms.

Concept	C# / .NET	Python	Qt / Andiya
App entry	`static void Main` + `Application.Run`	`app.exec()` in PyQt / Tk	`int main` + `QGuiApplication::exec()`
UI markup	XAML	Qt Designer .ui, or code	`.qml` files
Code-behind / VM	code-behind or ViewModel	slots on a QObject	C++ `QObject` subclasses
Data binding	`INotifyPropertyChanged`	property notify in PySide	`Q_PROPERTY` + `NOTIFY`
Events	`event EventHandler` / delegates	callbacks, signals in PyQt	signals and slots
Lists in UI	`ObservableCollection` + ItemsControl	list + Repeater-like widgets	`QAbstractListModel` + Repeater/ListView
Resources	resx / pack URIs	pkgutil data files	Qt resource system (`:/assets/...`)
Settings	user.config	QSettings or json	`QSettings` (registry/ini)
Threads	`Task` / `Dispatcher`	threads + queue	`QThread` + queued signals

C# developer shortcut

Treat QML as XAML with JavaScript expressions allowed in bindings. Treat every `Q_PROPERTY` as a bindable CLR property. Treat `signals:` as C# `event`. Treat `Q_INVOKABLE` as a public method the XAML can call. Treat `setContextProperty("player", &player)` as putting an object into `DataContext` with a well-known name.

Python developer shortcut

If you have seen PyQt5/PySide6, Andiya is the same object model, just written in C++. `QObject`, signals, slots, `QUrl`, `QImage` are identical types. QML is the extra language that replaces packing widgets in Python.

> **Pure C/C++ developer shortcut**
> Qt is not STL. Most interesting types inherit `QObject`, live on the heap, and participate in a parent-owned tree that deletes children automatically. Strings are `QString` (UTF-16). There is an event loop. You rarely write a raw Win32 message pump.

2.2 Modules Andiya actually links

From `CMakeLists.txt`:

```
find_package(Qt6 6.8 REQUIRED COMPONENTS
    Quick
    QuickControls2
    Multimedia
    Network
)
```

- **Quick** — the QML engine and scene graph (the renderer).
- **QuickControls2** — Button, Slider, Popup, ApplicationWindow, Menu…
- **Multimedia** — `QMediaPlayer`, `VideoOutput`, `QVideoSink`, audio devices.
- **Network** — catalog downloads in `StudioController`.

On Windows the deployed app also ships `ffmpegmediaplugin.dll` plus FFmpeg DLLs (`avcodec`, `avformat`, …). That is why RTSP can work without VLC and without a plugin of our own: protocol/demux lives under Qt Multimedia, not in Andiya's plugin ABI.

2.3 Two UI toolkits inside Qt — Andiya picked Quick

Qt Widgets	Qt Quick / QML
C++ classes: QPushButton, QMainWindow	Declarative files: Button {}, Rectangle {}
Like WinForms / classic desktop	Like WPF / mobile / custom chrome
Easier for dialog-heavy tools	Easier for animated, skinned players

Andiya uses Quick exclusively. There is no `QMainWindow`. The window is `ApplicationWindow` in QML, with `Qt.FramelessWindowHint` so the title bar is custom.

3. The Qt object model: QObject, properties, signals

Almost every C++ class that QML can see inherits `QObject` and starts with the macro `Q_OBJECT`. That macro enables Qt's meta-object compiler (**moc**). moc reads the header and generates extra C++ that implements:

- the class name as a string
- property get/set
- signal emission
- invokable methods

You never run moc yourself. CMake's `qt_standard_project_setup` wires it up. If you add a new `Q_PROPERTY` and the UI does not update, you usually forgot `Q_OBJECT`, forgot to emit the NOTIFY signal, or the header was not listed in the target so moc never saw it.

3.1 A tiny teaching example (not Andiya)

C++ header:

```
class Counter : public QObject {
    Q_OBJECT
    Q_PROPERTY(int value READ value WRITE setValue NOTIFY valueChanged)
public:
    explicit Counter(QObject *parent = nullptr) : QObject(parent) {}
    int value() const { return m_value; }
    void setValue(int v) {
        if (m_value == v) return;      // avoid extra notifications
        m_value = v;
        emit valueChanged();
    }
    Q_INVOKABLE void increment() { setValue(m_value + 1); }
signals:
    void valueChanged();
private:
    int m_value = 0;
};
```

Equivalent C# idea:

```
public class Counter : INotifyPropertyChanged {
    int _value;
    public int Value {
        get => _value;
        set { if (_value == value) return; _value = value; PropertyChanged?.Invoke(this,
            new PropertyChangedEventArgs(nameof(Value))); }
    }
    public event PropertyChangedEventHandler PropertyChanged;
    public void Increment() => Value++;
}
```

Equivalent Python (PySide6) idea:

```
class Counter(QObject):
    valueChanged = Signal()
    def __init__(self):
        super().__init__()
        self._value = 0
    def value(self):
        return self._value
    def setValue(self, v):
        if self._value == v: return
        self._value = v
        self.valueChanged.emit()
    value = Property(int, value, setValue, notify=valueChanged)
```

QML that uses it, once C++ did `engine.rootContext()->setContextProperty("counter", &counter)`:

```
Text { text: "Count: " + counter.value }
Button { text: "+1"; onClicked: counter.increment() }
```

When `increment()` runs, it emits `valueChanged`. The Text binding re-evaluates. You do not write `label.Text = ...` by hand. That is the same idea as WPF `{Binding Value}`.

3.2 Signals and slots

A **signal** is a declaration of "this happened". A **slot** (or any callable) is "when that happens, run me". Connection is not a C# multicast delegate you subscribe with `+=`; it is `QObject::connect`.

From Andiya `src/main.cpp`, when the capture folder changes, the screenshot list reloads:

```
QObject::connect(&player, &PlayerController::captureDirectoryChanged,
                 &screenshots, [&player, &screenshots] {
    screenshots.setDirectory(player.captureDirectoryPath());
});
```

That is: "player signal → lambda slot". Qt can also connect signal to a real member function by pointer-to-member, which is type-safe (the compiler checks the signatures).

> **Queued connections and threads**
> If the sender and receiver live on different threads, Qt can post the call onto the receiver's event loop (`Qt::QueuedConnection`). Andiya uses this so filter workers finish on a background thread, then hop back to the UI thread to present a frame. You do not marshal with `Control.Invoke`; you emit a signal or `QMetaObject::invokeMethod(..., Qt::QueuedConnection)`.

3.3 Q_PROPERTY in the real player

A shortened view of `src/player/PlayerController.h` — this is the contract QML binds to:

```
class PlayerController final : public QObject
{
    Q_OBJECT
    Q_PROPERTY(QString title READ title NOTIFY mediaChanged)
    Q_PROPERTY(bool playing READ playing NOTIFY playingChanged)
    Q_PROPERTY(qreal volume READ volume WRITE setVolume NOTIFY volumeChanged)
    Q_PROPERTY(qint64 position READ position NOTIFY positionChanged)
    Q_PROPERTY(qint64 duration READ duration NOTIFY durationChanged)
    Q_PROPERTY(bool isLiveStream READ isLiveStream NOTIFY mediaChanged)
    Q_PROPERTY(bool compareModeActive READ compareModeActive NOTIFY compareModeChanged)
    // ... dozens more
public:
    Q_INVOKABLE void openMedia(const QUrl &url);
    Q_INVOKABLE void openNetworkStream(const QString &urlText);
    Q_INVOKABLE void togglePlayback();
    Q_INVOKABLE void seek(qint64 positionMs);
    Q_INVOKABLE void enterCompareMode();
signals:
    void mediaChanged();
    void playingChanged();
    void notification(const QString &title, const QString &message);
};
```

QML then writes things like:

```
title: player.hasMedia ? player.title + " - Andiya" : "Andiya"

Slider {
    value: player.duration > 0 ? player.position / player.duration : 0
    enabled: player.hasMedia && !player.isLiveStream && player.duration > 0
    onMoved: player.seek(value * player.duration)
}
```

Read those as XAML bindings plus an event handler. `player` is not imported; it is a context property created in `main.cpp` (chapter 6).

3.4 Parent ownership (important)

If you pass a `QObject *parent` into a constructor, Qt deletes the child when the parent dies. Stack-allocated QObjects in `main()` (Andiya's pattern) live until `main` returns, which is after `app.exec()` finishes — that is fine. Heap objects without a parent must be deleted or they leak. This is closer to a WinForms control tree than to `shared_ptr` everywhere.

4. QML in one sitting (think XAML, not HTML)

QML is a declarative language. A file describes a tree of items. Curly braces are object definitions, not C scopes. `property` creates a bindable field. `id:` names a node so siblings can refer to it. JavaScript is allowed in expressions and handlers, but Andiya keeps logic in C++ whenever it touches files, codecs, or plugins.

4.1 A complete tiny window (teaching example)

```
import QtQuick
import QtQuick.Controls

ApplicationWindow {
    width: 400
    height: 200
    visible: true
    title: "Hello"

    Column {
        anchors.centerIn: parent
        spacing: 12
        Text { id: label; text: "Idle" }
        Button {
            text: "Play"
            onClicked: label.text = "Playing"
        }
    }
}
```

`onClicked` is the handler for the control's `clicked` signal. Naming is always `on` + capitalized signal name: `clicked` → `onClicked`, `valueChanged` → `onValueChanged`.

4.2 Anchors vs layouts

QML has two layout styles, and Andiya uses both.

Style	Like	Used when
`anchors.fill: parent`	DockStyle.Fill / HorizontalAlignment Stretch	A child should occupy its parent (video surface, overlays)
`RowLayout` / `ColumnLayout`	StackPanel / Grid with * rows	Toolbars, panels, dialogs that must reflow

Teaching snippet:

```
Item {
    anchors.fill: parent
    Rectangle {
        anchors.left: parent.left
        anchors.top: parent.top
        anchors.bottom: parent.bottom
        width: 72          // left rail
        color: "#18202B"
    }
    Rectangle {
        anchors.left: parent.left
        anchors.leftMargin: 72
        anchors.right: parent.right
        anchors.top: parent.top
        anchors.bottom: parent.bottom
        color: "#0B1119"   // remaining stage
    }
}
```

Andiya's main chrome in `qml/Main.qml` is a `RowLayout`: nav rail | library | media column | captures/tools. Fullscreen hides the docks by visibility, not by destroying them, so selection is kept.

4.3 Custom components = UserControls

A file named `IconButton.qml` becomes a type `IconButton` inside the same QML module. Properties you declare become the component's public API:

```
// qml/components/IconButton.qml (excerpt)
Rectangle {
    id: root
    property string glyph: ""
    property string label: ""
    property string toolTip: ""
    property bool active: false
    property color accentColor: Theme.cyan
    signal clicked()
    TapHandler { onTapped: root.clicked() }
}
```

Call site, anywhere in the app:

```
IconButton {
    glyph: "CMP"
    toolTip: "Compare original vs. filtered (K)"
    active: player.compareModeActive
    onClicked: player.compareModeActive ? player.exitCompareMode()
                                        : player.enterCompareMode()
}
```

That is a UserControl with dependency properties, plus a custom routed event named `clicked`.

4.4 Singletons

`qml/Theme.qml` and `qml/TimeFormat.qml` are registered as singletons in CMake:

```
set_source_files_properties(qml/Theme.qml PROPERTIES
    QT_QML_SINGLETON_TYPE TRUE
    QT_RESOURCE_ALIAS Theme.qml
)
```

Any QML file can then write `Theme.cyan` or `TimeFormat.time(player.position)` without creating an instance. This is a ResourceDictionary + static helper in one object. `pragma Singleton` at the top of the QML file is the matching declaration.

4.5 JavaScript in QML — keep it thin

You *can* write functions in QML:

```
function toggleFullscreen() {
    if (visibility === Window.FullScreen) showNormal()
    else showFullScreen()
}
```

Andiya does that for window chrome and shortcut gating. It does *not* decode video or talk to Python from QML. If a feature needs a file path, a process, or a pixel buffer, it belongs in C++.

4.6 List models

C# `ItemsControl` + `ObservableCollection` becomes QML `Repeater` / `ListView` + `QAbstractListModel`.

C++ must implement `rowCount`, `data`, and `roleNames` (role names become the property names in the delegate). `PluginListModel` exposes `name`, `vendor`, `pluginEnabled`, and so on. QML:

```
Repeater {
    model: pluginModel
    RowLayout {
        required property int index
        required property string name
        required property bool pluginEnabled
        Switch {
            checked: pluginEnabled
            onToggled: pluginModel.setPluginEnabled(index, checked)
        }
    }
}
```

When C++ calls `beginMoveRows` / `endMoveRows` (filter reorder), the Repeater moves items. When it emits `filterStackChanged`, other UI (the filter graph, compare view) refreshes.

4.7 Imports you will see at the top of Andiya QML

Import	Gives you
`QtQuick`	Item, Rectangle, Text, Image, Timer, bindings
`QtQuick.Controls`	ApplicationWindow, Button, Slider, Popup, Switch
`QtQuick.Layouts`	RowLayout, ColumnLayout, Layout.fillWidth
`QtQuick.Dialogs`	FileDialog
`QtMultimedia`	VideoOutput
`QtQuick.Effects`	MultiEffect (brightness/contrast overlay)

5. Andiya's folder map and build

5.1 Source tree

Path	Role
`src/main.cpp`	Process entry, construct controllers, load QML
`src/player/`	Playback, frames, exports, screenshots, thumbnails, studio
`src/plugins/`	Plugin model, validation, filter pipeline, Python host
`include/andiya/plugin_api.h`	Public C ABI for native filters
`qml/`	Window, dialogs, Theme
`qml/components/`	Buttons, menus, media stage, timeline, compare view
`qml/panels/`	Library, Tools, captures, filter graph dialog
`plugins/`	Example native + Python filters shipped with the app
`runtime/python/`	Host script spawned per Python plugin
`docs/`	User guide, building, plugin reference — and this PDF
`CMakeLists.txt`	Targets, QML module, copy plugins next to the exe

5.2 How a QML file becomes part of the exe

Andiya does not load QML from loose files next to the exe in production. CMake compiles them into a Qt resource module named URI `Andiya`:

```
qt_add_qml_module(Andiya
    URI Andiya
    VERSION 1.0
    QML_FILES
        qml/Main.qml
        qml/Theme.qml
        qml/components/MediaStage.qml
        ...
)
```

Startup then does:

```
engine.loadFromModule(QStringLiteral("Andiya"), QStringLiteral("Main"));
```

That means: "find type Main inside module Andiya". Adding a new QML file is a three-step ritual:

1. Create the `.qml` file.

2. Add `set_source_files_properties(... QT_RESOURCE_ALIAS ...)` if you care about the name inside the module.
3. List it under `QML_FILES` in `CMakeLists.txt`.

If you forget step 3, the engine cannot see the type. If you change QML only, you still need a rebuild so the resource compiler (rcc) and qmlcache pick it up. That is why "I edited QML but the running exe looks old" is usually a stale build, not a QML bug.

5.3 CMake vs a Visual Studio C# project

`qt_add_executable(Andiya ...)` is "here are the .cpp/.h files". `target_link_libraries(Andiya PRIVATE Qt6::Quick Qt6::Multimedia ...)` is package references. Extra targets exist for:

- `AndiyaSoftContrastPlugin` — sample native filter DLL
- `AndiyaFilterWorker` — out-of-process native worker
- `AndiyaPluginValidator` — package/manifest checks

POST_BUILD copy commands place plugins and `andiya_python_host.py` next to `Andiya.exe` so a developer run works without an installer.

5.4 Why windeployqt exists

Unlike a fully statically linked C# app, a Qt exe needs sibling DLLs: Qt6Quick.dll, platform plugin `qwindows.dll`, QML modules, multimedia plugins. `windeployqt` copies that tree into `build/Release`. If the exe starts and instantly exits, missing platform plugins are a common cause.

6. Boot sequence: from main() to the window

Read `src/main.cpp` top to bottom. This is the entire composition root.

6.1 Create the application object

```
QGuiApplication::setApplicationName("Andiya");
QGuiApplication::setOrganizationName("Andiya");
QGuiApplication app(argc, argv);
QQuickStyle::setStyle("Basic");
```

`QGuiApplication` is the process-wide object: event loop, high-DPI, clipboard, screens. Organization/application names are not cosmetic — `QSettings` uses them as the registry path (Windows) or config folder (others). Style "Basic" means Andiya draws its own controls instead of native Fusion/Windows chrome.

Two special argv modes exist: `--self-test DIR` runs integration tests without the UI, and `--ui-smoke DIR` opens Workspace tabs and screenshots them. Ordinary users never pass those.

6.2 Construct the C++ "view models"

```
PluginListModel plugins;
PlayerController player(&plugins);
StudioController studio(&player, &plugins);
ThumbnailProvider thumbnails;
ScreenshotListModel screenshots;
```

Order matters: the player needs the plugin model to apply filters. Studio needs both, because a workspace export includes playlist + filter presets. Screenshots wait 250 ms before scanning the capture folder so the window can appear first.

6.3 Publish them to QML

```
QQmlApplicationEngine engine;
engine.addImageProvider("frames", new FrameStore);
engine.rootContext()->setContextProperty("studio", &studio);
engine.rootContext()->setContextProperty("player", &player);
engine.rootContext()->setContextProperty("pluginModel", &plugins);
engine.rootContext()->setContextProperty("thumbnails", &thumbnails);
engine.rootContext()->setContextProperty("screenshots", &screenshots);
engine.loadFromModule("Andiya", "Main");
return app.exec();
```

> **Context properties are globals on purpose**
> Every QML file can say `player.togglePlayback()` without imports. That is convenient for a single-window app and slightly implicit. A larger Qt codebase might register types with `qmlRegisterType` and instantiate them in QML instead. Andiya's size makes the global objects a fair trade.

`FrameStore` is a `QQuickImageProvider`. QML `Image { source: "image://frames/some-key" }` asks C++ for a `QImage`. Compare mode uses this kind of in-memory / cache-file round trip so both panes can show PNG snapshots without inventing a new control.

6.4 Command-line open

After the window loads, if argv has a path or URL, `player.openMedia` runs on a zero-delay timer (so QML bindings exist first). A second argument is treated as a subtitle file. This is the same idea as associating the exe with video files in the shell.

6.5 What `app.exec()` actually is

It is the message loop. Until it returns, Qt delivers mouse/keyboard, timers, queued signals, and vsync frames. C# `Application.Run` and Python `app.exec()` are the same idea. Long work on this thread freezes the UI — that is why filters run on `FilterPipeline`'s worker thread and child processes.

7. The window, theme, and reusable components

7.1 Main.qml is the shell

`qml/Main.qml` declares an `ApplicationWindow` with no native title bar:

```
ApplicationWindow {
    id: window
    flags: Qt.Window | Qt.FramelessWindowHint
           | Qt.WindowSystemMenuHint | Qt.WindowMinimizeButtonHint
           | Qt.WindowMaximizeButtonHint | Qt.WindowCloseButtonHint
    font.pixelSize: 14 * studio.textScale
    title: player.hasMedia ? player.title + " - Andiya" : "Andiya"
}
```

Because it is frameless, the file also implements drag-to-move, custom min/max/close buttons, and resize edges. That is extra QML compared with a normal OS window, and it is why the chrome can follow Theme colors.

Window-level properties keep sidebar state:

```
property bool pluginsOpen: false
property bool screenshotsOpen: false
property bool libraryOpen: true
readonly property bool dualSidebars: width >= 1280
```

On a wide window, library and Tools/Captures can both stay open. On a narrow window, opening one closes the other so the timeline never slides under a dock. That logic is UI-only; C++ does not know about sidebars.

7.2 Theme.qml — a palette object

Theme is a singleton `QtObject` with `styleName` and a big `colors` map. The rest of the UI reads `Theme.text`, `Theme.cyan`, `Theme.surface`. Changing the combo box in Settings/Workspace assigns `Theme.styleName = currentText`. A `Connections` block in Main writes it back to the player so `QSettings` remembers it:

```
Connections {
    target: Theme
    function onStyleNameChanged() { player.setThemeName(Theme.styleName) }
}
```

> **WPF analogue**
> DynamicResource brushes in a merged dictionary, plus a setting that picks which dictionary is active. There is no XAML `BasedOn` style hierarchy here; components read Theme properties directly.

7.3 Component catalog

File	Job
`GlassPanel.qml`	Rounded rectangle used as a card/panel
`IconButton.qml`	The app's button: glyph, label, tooltip, accent, keyboard focus
`ThemedMenu*.qml`	Menus that obey Theme (native Menu is unreadable on custom palettes)
`NavRailButton.qml`	Left-rail icon + caption
`MediaStage.qml`	VideoOutput, image, empty state, context menu, CompareView host
`PlaybackOverlay.qml`	Auto-hiding fullscreen OSD
`TimelineControls.qml`	Seek bar, volume, transport, overflow menu, LIVE badge
`CompareView.qml`	Side-by-side original / filtered export UI
`DisplayAdjustments.qml`	Brightness/contrast sliders (GPU MultiEffect, not a plugin)

7.4 Dialogs are Popup, not Window

Settings, About, Shortcuts, Network stream, Filter graph, Workspace: all `Popup` items parented to `Overlay.overlay`. That keeps them modal inside the same OpenGL/scene-graph window. Escape handling in Main closes the topmost popup, then exits fullscreen, then does nothing aggressive.

7.5 Shortcuts

Main.qml declares `Shortcut { sequence: "Space"; ... }` objects. Letter keys go through `plainShortcutEnabled()`, which returns false when a text field has a `cursorPosition` (so typing in the network-URL box does not capture a frame). StudioController can remap common actions; the dialog in Workspace edits that map.

The Shortcuts dialog (`?` / F1) is only documentation of the default set plus a few extras. Making shortcuts visible was a product requirement: people should not have to memorize S, K, comma, period.

8. Playback: files, streams, frames, audio

8.1 Opening media

`PlayerController::openMedia(const QUrl &url)` is the single entry for local files, images, and network URLs. Rough flow:

1. Save resume position of the previous file.
2. Bump an internal `m_generation` counter so in-flight filter previews are discarded.
3. Decide image vs audio vs video from the suffix (and live vs VOD from the URL scheme).
4. Images: load `QImage`, stop the media player.
5. AV: `m_player->setSource(url); m_player->play();`
6. Emit `mediaChanged` so the title, stage, and library all refresh.

Network URLs are not a second player. `openNetworkStream` only validates the typed string into a `QUrl` and calls `openMedia`. Live schemes (`rtsp`, `rtmp`, `udp`, …) set `isLiveStream`, which disables seeking and shows a LIVE badge. HTTP(S) is left seekable because it is often a file on a server, not a camera.

> **Plugins cannot add RTSP**
> The plugin ABI receives already-decoded BGRA frames. Demuxing and network I/O happen inside Qt Multimedia's FFmpeg plugin. New transports require a Qt/FFmpeg build that implements them, plus UI to pass the URL in — which Andiya already has.

8.2 How video reaches the screen

QML hosts a `VideoOutput` in `MediaStage.qml`:

```
VideoOutput {
    id: videoOutput
    anchors.fill: parent
    visible: player.hasMedia && !player.imageMode && player.mediaKind === "Video"
    fillMode: VideoOutput.PreserveAspectFit
    Component.onCompleted: player.attachVideoOutput(videoOutput)
}
```

C++ then steals the output's `QVideoSink`:

```
void PlayerController::attachVideoOutput(QObject *videoOutput)
{
    if (!videoOutput) return;
    m_displaySink = videoOutput->property("videoSink").value<QVideoSink *>();
    if (m_displaySink && m_currentFrame.isValid())
        presentFrame(m_currentFrame);
}
```

Decoded frames do not always go straight to that sink. `handleDecodedFrame` stores the frame, optionally waits for a filtered preview, then calls `presentFrame` which is just `m_displaySink->setVideoFrame(frame)`.

```
QMediaPlayer  --decodes-->  QVideoFrame
        |
        v
handleDecodedFrame()
        |-- no live filters --> presentFrame() --> VideoOutput
        |-- live filters ------> FilterPipeline (bg thread, child processes)
                                    |
                                    +--> QImage --> QVideoFrame --> presentFrame()
```

Figure 2. Display path. Preview is scaled to max 960 px so the UI stays responsive. Export uses a second pipeline at full resolution.

8.3 Frame step uses timestamps, not “duration / fps”

Comma and period do not assume 24 or 30 fps. The player pauses, notes the current frame’s `startTime`, then keeps decoding until a neighbouring timestamp appears (a look-behind window for backward). If that times out, it reports failure instead of guessing. That is slower than VLC’s decoded-index step on some files, but it stays honest on variable-frame-rate video.

8.4 Audio

Qt can play audio through `QMediaPlayer` directly. Andiya also uses `QAudioBufferOutput` + `QAudioSink` so it can restart the sink on device changes and recover from some backend stalls. Volume is a `qreal` that can exceed 1.0 (soft boost); the UI shows a percentage and turns amber above 100%.

8.5 Subtitles

External SRT/VTT-style files are parsed in C++ into a vector of `{startMs, endMs, text}`. On position changes, the controller picks the active cue and exposes `subtitleText`. QML only draws a bubble. Timing delay, font size, color and vertical position are properties persisted in settings. Embedded tracks from the container, when Qt exposes them, appear in the track lists.

8.6 Images vs video

Still images skip `QMediaPlayer`. They load into `m_originalImage` and show in an `Image {}` item. Filters still apply (compare mode, capture). This is why PNG/JPEG/WEBP work without a decoder timeline.

9. The filter pipeline and plugin system

This is the part that makes Andiya more than a thin Qt demo. Filters are not compiled into the player. They are discovered at runtime, ordered in a list, and executed out of the player process.

9.1 What a plugin is allowed to do

A plugin receives one raster frame and must return a frame of the **same width and height**. Today the working format is BGRA32 (8-bit, pre-not-required, alpha left alone unless the plugin touches it). Plugins cannot:

- open files as a media source (no RTSP plugin)
- resize the canvas / AI upscale via the current ABI
- draw widgets into the QML tree

They can: color grade, denoise, grayscale, overlay math on pixels, call Python libraries inside their own process, crash without taking Andiya down.

9.2 Discovery and the Tools panel

`PluginListModel` scans plugin directories, reads `plugin.json`, records native vs Python, and implements `QAbstractListModel` so QML can list cards. Enable, strength, numeric parameters, up/down reorder, and named presets all live here. `snapshot()` produces the ordered list the worker thread will run.

The Tools drawer visualizes that order as Source → filter → filter → Output. The expanded Filter Graph dialog wraps long chains. Reorder buttons call `moveFilterUp` / `moveFilterDown`, which emit `filterStackChanged`. Compare mode listens to that signal so toggling a plugin updates the filtered pane without closing K mode.

9.3 Two pipelines: preview vs capture

`PlayerController` owns:

```
FilterPipeline *m_previewPipeline;
FilterPipeline *m_capturePipeline;
```

Each pipeline is a `QObject` living with a dedicated `QThread`. `process(image, timestamp, stack, callback)` posts work to that thread. Preview scales down to 960 px and may skip frames if the previous preview is still busy (`m_skippedPreviews` in diagnostics). Capture keeps full resolution and fails the export if the worker errors, instead of silently showing a stale preview.

Strength blending happens in C++ after the plugin returns: output = lerp(original, filtered, strength). A plugin that always produces a full effect can still be mixed 30% in the UI.

9.4 Native plugins (C ABI)

Header `include/andiya/plugin_api.h` is a C API so C, C++, and even Rust can implement it. The exported function is:

```
ANDIYA_PLUGIN_EXPORT int32_t ANDIYA_PLUGIN_CALL
andiya_plugin_initialize(
    uint32_t host_api_version,
    const AndiyaHostApi *host,
    AndiyaImageFilterApi *plugin);
```

The plugin fills `AndiyaImageFilterApi` with function pointers: `plugin_id`, `process_frame`, `destroy`. `process_frame` gets input/output `AndiyaFrame` structs (pointer, width, height, stride, timestamp).

The shipped example `plugins/soft-contrast/plugin.cpp` walks every pixel and applies contrast/warmth. That is ordinary C++ image processing; Qt is only used there for JSON parameter parsing.

Andiya does **not** `LoadLibrary` that DLL into the UI process for preview/export. `AndiyaFilterWorker.exe` loads it. Framed binary I/O plus a timeout means a native access violation kills the worker, Andiya bypasses the frame, and playback continues.

9.5 Python plugins (out of process)

For each enabled Python filter, C++ starts:

```
py -3  python_runtime/andiya_python_host.py  path/to/filter.py
```

The host script (`runtime/python/andiya_python_host.py`) imports your module and loops on stdin. Wire format (little-endian, no JSON per frame):

Direction	Header	Then
Andiya → plugin	magic ANFR, w, h, stride, format, timestamp, size	raw pixels
plugin → Andiya	magic ANRS, status, w, h, stride, format, size	raw pixels if status==0

Your module only implements:

```
def process_frame(width, height, stride, pixel_format, timestamp_us, data):
    # data is BGRA bytes, stride * height long
    buffer = bytearray(data)
    ...
    return bytes(buffer)   # or None to bypass
```

The grayscale example averages channels with video luma coefficients and leaves alpha alone. You may import NumPy/OpenCV inside that process; they never link into Andiya.exe.

> **Why not in-process Python?**
> Embedding CPython in a Qt GUI means GIL fights, crash-the-app bugs, and matching MSVC runtimes. A child process is slower per frame and much safer. Preview downscaling exists specifically because Python filters can be heavy.

9.6 Manifest

Each plugin folder has `plugin.json`: id, name, vendor, version, type (native/python), entry (dll or py), capabilities, parameter schema. Validation lives in `PluginValidation.cpp`. Installers can be a folder copy or a signed bundle; Windows can show publisher certificate details. Isolation is about crashes, not an OS sandbox — a plugin process can still read the disk and network.

10. Library, captures, compare mode, workspace

10.1 Library and playlists

Recent files and the playlist are `QVariantList` properties on the player, persisted with `QSettings`. `MediaLibraryPanel.qml` is a QML list. Video rows ask `ThumbnailProvider` for a poster: a hidden `QMediaPlayer` seeks, grabs one frame, writes a JPEG cache. Live streams skip that (no local file, no duration).

10.2 Captures panel

`ScreenshotListModel` watches the capture directory with `QFileSystemWatcher`. It exposes latest-file metadata (size, dimensions, time) and supports remove-from-list vs delete-on-disk vs clear-all. Hidden paths are remembered in settings so "remove from app" survives a rescan.

Filenames are unique and include timestamp plus original/filtered plus a UUID. Writes use `QSaveFile` (atomic replace) so a crash mid-write does not leave a truncated PNG as the latest capture.

10.3 Compare mode

Shortcut `K` / CMP button calls `enterCompareMode()`. The player pauses if needed, grabs the current decoded image, writes cache PNGs (original + filtered), and exposes two URLs. `CompareView.qml` is an overlay on the media stage:

- left: untouched frame, Save
- right: pipeline output, Save (disabled if no filter is on)
- filmstrip: play/pause, previous/next frame, Live preview timer (~350 ms) while playing

Live preview is throttled so a Python chain is not invoked every 16 ms. Export always goes through the capture pipeline at full resolution, not the 960 px preview.

10.4 Bookmarks, A–B loop, batch export

Bookmarks are a list of {position, note} on the player. A–B loop sets start/end timestamps and seeks back when playback crosses the end. Batch capture / bookmark export / image-folder export are queued in `ExportQueue` (cap 200 items) with progress properties QML binds to. Cancel keeps files already written.

10.5 Workspace

`StudioController` is the "project / preferences" object: text scale, shortcut map, encrypted stream profiles (Windows DPAPI), plugin catalog HTTP client, workspace JSON import/export. The Workspace dialog is a tabbed Popup. Portable workspace files deliberately strip URL credentials and query strings — signed camera URLs should live in profiles, not in a file you email.

11. Persistence, threading, and diagnostics

11.1 QSettings

`QSettings` is a key-value store. On Windows this is typically the registry under HKCU\Software\Andiya\Andiya. Self-tests switch organization name so they cannot clobber your real settings.

Typical keys the player writes: theme, window geometry, volume, recent media, playlist, resume positions, capture folder, subtitle appearance, bookmarks, filter enable/order/strength/presets.

> **.NET analogue**
>
> `Properties.Settings.Default` or a small JSON file. There is no Entity Framework here. Lists are stored as `QVariantList` which Qt serializes for you.

11.2 Threading rules Andiya follows

Work	Thread
QML, PlayerController public methods	GUI / main
QMediaPlayer decode callbacks	Qt multimedia (then hop to player)
Filter pixel I/O	FilterPipeline worker thread + child process
PNG compress/write	QThreadPool global instance
Thumbnails	ThumbnailProvider's internal player

Never create a `QProcess` on thread A and destroy it on thread B. FilterPipeline uses `thread_local` hosts on the worker and `moveToThread` so ownership stays put. Completion lambdas use `QPointer` guards so a destroyed PlayerController does not get called after Close.

11.3 Diagnostics

`player.diagnostics()` returns a map: backend name, codecs from metadata, buffer %, decoded frame count, skipped previews, last filter error. Qt does not expose "which DXVA decoder is active" through the APIs Andiya uses; the UI says that honestly instead of inventing a GPU name.

12. How to change the app yourself

Four recipes. Each assumes you can rebuild (`cmake --build build --config Release`) and that you re-run the new exe, not an old shortcut.

12.1 Add a QML-only button that calls existing C++

Goal: a button that captures the original frame.

1. Open `qml/components/TimelineControls.qml`.
2. Next to the other `IconButton`s, add:

```
IconButton {
    glyph: "PNG"
    toolTip: "Save untouched frame (Ctrl+Shift+S)"
    enabled: player.hasMedia
    onClicked: player.captureOriginalFrame()
}
```

No C++ change. `captureOriginalFrame` is already `Q_INVOKABLE`.

12.2 Add a new C++ property and bind it

Goal: show the number of decoded frames in the timeline.

In `PlayerController.h`:

```
Q_PROPERTY(int decodedFrames READ decodedFrames NOTIFY diagnosticsChanged)
// ...
[[nodiscard]] int decodedFrames() const { return int(m_decodedFrames); }
```

`m_decodedFrames` already exists and `diagnosticsChanged` already fires from the preview callback. In QML:

```
Text {
    text: "frames " + player.decodedFrames
    color: Theme.textMuted
    font.pixelSize: 9
}
```

Rebuild. If QML says the property is undefined, moc did not see the header (clean rebuild) or the name is misspelled (QML is case-sensitive).

12.3 Add a shortcut

In `qml/Main.qml` next to the other Shortcut objects:

```
Shortcut {
    sequence: "Ctrl+L"
    onActivated: window.openLibraryPanel("Library")
}
```

Also add a row to `qml/ShortcutsDialog.qml` so it appears in the cheat sheet. If the key is a letter, consider `enabled: window.plainShortcutEnabled()` so it does not fire while typing a URL.

12.4 Add a Python filter

1. Create a folder with `plugin.json` and `filter.py`.
2. Implement `process_frame(...)` as in `plugins/python-grayscale/filter.py`.
3. In the app: Tools → install plugin folder (or copy next to the exe under `plugins/` and reload).
4. Enable it. Open Compare (K) to see original vs filtered. Save PNG from either pane.

Minimal invert filter (teaching):

```
def process_frame(width, height, stride, pixel_format, timestamp_us, data):
    if pixel_format != 1:
        return None
    out = bytearray(data)
    for y in range(height):
        row = y * stride
        for x in range(width):
            i = row + x * 4
            out[i] = 255 - out[i]         # B
            out[i+1] = 255 - out[i+1]     # G
            out[i+2] = 255 - out[i+2]     # R
    return bytes(out)
```

Native version: copy `docs/examples/native-invert` or `plugins/soft-contrast`, keep `andiya_plugin_initialize` exported, process BGRA in place or into the provided output buffer, return 0 on success.

12.5 Where not to put new features

Feature idea	Put it here
New button, layout, theme color	QML + Theme.qml
New playback behaviour	PlayerController
New plugin setting	PluginListModel + Tools QML
New network catalog field	StudioController
New pixel algorithm	A plugin, not the player
New streaming protocol	Qt Multimedia / FFmpeg, plus a URL in openMedia — not a plugin

13. Glossary and further reading

13.1 Glossary

Term	Meaning in this codebase
QObject	Base class with signals, properties, parent ownership
moc	Meta-object compiler; generated from Q_OBJECT headers
Q_PROPERTY	Bindable field QML can read/write
Q_INVOKABLE	C++ method QML can call
signal / slot	Event and handler, possibly cross-thread
context property	Named C++ object injected into all QML
QML module	Compiled set of .qml files under a URI (Andiya 1.0)
VideoOutput	QML item that displays frames via a QVideoSink
QVideoSink	C++ object you push QVideoFrame into
FilterPipeline	Background serial worker that runs the plugin stack
ABI plugin	Native shared library exporting andiya_plugin_initialize
Python host	andiya_python_host.py speaking ANFR/ANRS frames on stdio
QSettings	Persisted key-value user state
windeployqt	Copies Qt runtime files next to the exe

13.2 Suggested Qt reading order (official docs)

1. Qt QML tutorial — “Your First QML Application”
2. The QObject model and signals & slots
3. Qt Quick Controls
4. Qt Multimedia QMediaPlayer / VideoOutput
5. QAbstractListModel for QML
6. Resource system and qt_add_qml_module

You do not need Qt Widgets, Qt Network (beyond Studio catalogs), or Qt 3D to understand Andiya.

13.3 Files to open in this order when exploring the repo

1. `src/main.cpp` — who exists
2. `qml/Main.qml` — how the shell is laid out

3. `src/player/PlayerController.h` — every UI-facing playback API
4. `qml/components/MediaStage.qml` — video + empty state + compare host
5. `qml/components/TimelineControls.qml` — transport bindings
6. `src/plugins/PluginListModel.h` and `FilterPipeline.cpp`
7. `include/andiya/plugin_api.h` plus one example plugin
8. `PLUGIN_DEVELOPMENT.md` when you write your own filter

13.4 A last C#/Python picture of one click

User presses Space.

1. QML `Shortcut` runs `player.togglePlayback()` (like calling a ViewModel command).
2. C++ `togglePlayback` calls `QMediaPlayer::pause` or `play`.
3. The player eventually emits state changes; Andiya emits `playingChanged`.
4. Every QML `player.playing` binding updates: the glyph on the play button flips, the overlay play icon changes, compare live-preview timer may start.
5. If live filters are on, each new decoded frame is scaled, sent to a child process, blended by strength, and pushed into the VideoOutput sink.

That loop — **input → invokable → Qt backend → signal → binding** — is the entire application, repeated for Open, Seek, Capture, Tools, Workspace, and streams.